

Linear Algebra

A complete course workbook

Layer 1 • Topic 1 • AI & LLM Learning Roadmap

6 lessons

Vectors through eigenvalues

35 exercises

Starter, Standard, Challenge

3 weeks

~1.5 hrs/day

Python throughout

NumPy + SciPy

How to use this workbook

This workbook is a complete course for linear algebra — the first topic in Layer 1 of the AI & LLM roadmap. It is designed to be worked through in order, one lesson per sitting.

Structure of each lesson

Concept — plain English explanation of the idea

Why it matters — direct connection to ML and TurboQuant

The math — formal definitions and notation

Worked examples — step by step with full working

Python code — runnable NumPy implementations

Exercises — Starter (warm-up), Standard (core skill), Challenge (harder)

You will need Python with NumPy installed. Every code block in this workbook is runnable as-is. Type the code yourself rather than copy-pasting — the act of typing it helps retention.

Before you begin

Install Python 3.10+ and NumPy: `pip install numpy scipy matplotlib`
Keep a notebook alongside this workbook for working through the math by hand.
Do not skip exercises. The exercises are where the learning happens.
If an exercise takes more than 30 minutes, move on and return to it later.

Lesson 1.1 Vectors

What they are, how they behave, why everything in ML is a vector

Estimated time: 3-4 hours

The concept

A vector is simply a list of numbers. That is it. In 2D, a vector like $[3, 4]$ points 3 units along the x-axis and 4 units along the y-axis. In 1536 dimensions, a vector is a list of 1536 numbers.

In machine learning, vectors represent almost everything:

- A word embedding: $[0.23, -1.45, 0.87, \dots, 0.12]$ — 1536 numbers encoding meaning
- An image: $[255, 128, 64, \dots, 0]$ — one number per pixel per colour channel
- A model weight: just another array of numbers
- A token in a transformer: a vector that gets transformed at each layer

Connection to TurboQuant

TurboQuant's entire job is to compress vectors.

The input is a vector x in $\mathbb{R}^d$ (d -dimensional space).

The output is a compact binary string that approximately preserves x .

All the theory in the paper — distortion, inner products, norms — is about vectors.

You cannot read a single equation in the paper without understanding vectors.

The math

Definition: Vector

A vector x in $\mathbb{R}^d$ is an ordered list of d real numbers: $x = [x_1, x_2, \dots, x_d]$. The number d is called the dimension.

Notation you will see in papers and code:

- x (bold lowercase) = a vector
- x_i = the i -th element of vector x (also written $x[i]$ in code)
- $\mathbb{R}^d$ = the set of all d -dimensional real-valued vectors
- x in $\mathbb{R}^d$ = ' x is a d -dimensional vector'

Vector addition

Add two vectors of the same dimension by adding element-by-element:

$$x + y$$

$x = [1, 2, 3]$, $y = [4, 5, 6]$
 $x + y = [1+4, 2+5, 3+6] = [5, 7, 9]$

Geometrically: place y's tail at x's tip. The sum is the vector from origin to y's tip.

Scalar multiplication

Multiplying a vector by a number (scalar) scales all its elements:

$c * x$

$x = [1, 2, 3]$, $c = 2.5$
 $2.5 * x = [2.5, 5.0, 7.5]$

Geometrically: stretches ($c > 1$) or shrinks ($0 < c < 1$) the vector. Negative c reverses direction.

The L2 norm (length)

Definition: L2 norm

The L2 norm (Euclidean length) of a vector x is: $\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2}$.
This is the straight-line distance from the origin to x .

Example: $x = [3, 4]$. $\|x\| = \sqrt{9 + 16} = \sqrt{25} = 5$.

The L2 norm appears constantly in ML — in loss functions (mean squared error), in distance calculations, and in TurboQuant's distortion formula $D_{\text{mse}} = E[\|x - \hat{x}\|^2]$.

Unit vectors and normalization

Definition: Unit vector

A vector x is a unit vector if $\|x\| = 1$. To normalize any vector: $x_{\text{unit}} = x / \|x\|$.

TurboQuant assumes all inputs are unit vectors (x in S^{d-1} , the unit hypersphere). For non-unit vectors, you normalize before quantizing and store the norm separately, then rescale at dequantization time.

Python: working with vectors

```
import numpy as np

# Create vectors
x = np.array([1.0, 2.0, 3.0])
y = np.array([4.0, 5.0, 6.0])
```

```

# Vector addition
z = x + y
print(z)  # [5. 7. 9.]

# Scalar multiplication
w = 2.5 * x
print(w)  # [2.5 5. 7.5]

# L2 norm
norm_x = np.linalg.norm(x)
print(norm_x)  # 3.7417...

# Verify manually
manual_norm = np.sqrt(x[0]**2 + x[1]**2 + x[2]**2)
print(np.isclose(norm_x, manual_norm))  # True

# Normalize to unit vector
x_unit = x / np.linalg.norm(x)
print(np.linalg.norm(x_unit))  # 1.0

# High-dimensional vectors work identically
d = 1536
v = np.random.randn(d)  # random 1536-dim vector
v_unit = v / np.linalg.norm(v)
print(np.linalg.norm(v_unit))  # 1.0

```

Worked example: normalizing an embedding

Say a language model produces the embedding $x = [0.6, 0.8, 0.0]$ for the word 'cat'. Before we can use TurboQuant, we normalize it.

Step 1: Compute the norm. $\|x\| = \sqrt{0.36 + 0.64 + 0.0} = \sqrt{1.0} = 1.0$.

It is already a unit vector. Now try $x = [3.0, 4.0, 0.0]$:

$\|x\| = \sqrt{9 + 16 + 0} = 5.0$. $x_{\text{unit}} = [3/5, 4/5, 0] = [0.6, 0.8, 0.0]$.

Check: $\|x_{\text{unit}}\| = \sqrt{0.36 + 0.64 + 0} = 1.0$. Correct.

Exercises 1.1

No.	Difficulty	Exercise
1.1.1	Starter	Create a NumPy vector $[2, 5, -3, 1]$. Print its shape, its third element, and its norm.
1.1.2	Starter	Add vectors $[1,0,0]$, $[0,1,0]$, and $[0,0,1]$. What do you get? Why?
1.1.3	Standard	Write a function <code>normalize(v)</code> that returns the unit vector in the same direction as v . Do not use <code>np.linalg.norm</code> — compute the norm manually. Test it on five random vectors and verify each output has norm = 1.0.
1.1.4	Standard	Generate a random vector in 512 dimensions. Normalize it. Verify that its norm is exactly 1.0 (use <code>np.isclose</code> with <code>atol=1e-6</code>).

1.1.5	Standard	Write a function that takes two unit vectors and returns True if they are 'nearly the same direction' (i.e., their angle is less than 10 degrees). Hint: use the dot product formula from Lesson 1.2 — come back to this after completing that lesson.
1.1.6	Challenge	Generate 10,000 random unit vectors in 100 dimensions. For each vector, print only the first coordinate. Plot a histogram of these first coordinates using matplotlib. What distribution do they follow? Why? (Hint: look up the multivariate normal distribution and think about what constraint living on the unit sphere imposes.)

Lesson 1.2 The Dot Product

The single most important operation in all of machine learning

Estimated time: 3-4 hours

The concept

The dot product takes two vectors and returns a single number. It measures how much two vectors 'agree' — how much they point in the same direction. If the dot product is large and positive, the vectors are pointing roughly the same way. If it is zero, they are perpendicular. If it is negative, they point away from each other.

In ML, dot products appear in:

- Attention scores in transformers: $\text{score}(q, k) = q \cdot k$
- Similarity search: finding the nearest vector to a query
- Linear layers: $\text{output} = W @ x$ (a matrix of dot products)
- TurboQuant's inner product quantizer: designed to preserve $\langle y, x \rangle$ after compression

Why TurboQuant cares so deeply about dot products

In nearest-neighbor search (vector databases), you find similar items by dot product. When a user asks a question, you embed it, then find vectors with the highest dot product. If your compression distorts dot products, your search returns wrong results. TurboQuant's Algorithm 2 is specifically designed to preserve dot products without bias. Theorem 2 proves: $E[\langle y, \text{decompress}(\text{compress}(x)) \rangle] = \langle y, x \rangle$ exactly.

The math

Definition: Dot product

The dot product of two vectors x and y of the same dimension d is: $\langle x, y \rangle = x \cdot y = x_1 y_1 + x_2 y_2 + \dots + x_d y_d$. The result is a single real number.

The geometric interpretation

$$x \cdot y = \|x\| * \|y\| * \cos(\theta)$$

where θ is the angle between x and y .

If $\theta = 0$ (same direction): $x \cdot y = \|x\| * \|y\|$ (maximum)

If $\theta = 90$ (perpendicular): $x \cdot y = 0$

If $\theta = 180$ (opposite): $x \cdot y = -\|x\| * \|y\|$ (minimum)

For unit vectors ($\|x\| = \|y\| = 1$), the dot product simplifies to just $\cos(\theta)$. This means the dot product is a direct measure of the cosine of the angle between them.

Cosine similarity

Definition: Cosine similarity

$\text{cosine_sim}(x, y) = (x \cdot y) / (\|x\| * \|y\|)$. This measures similarity independent of magnitude. Equal to $\cos(\theta)$, ranging from -1 (opposite) to 1 (identical direction).

Python: dot products and similarity

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([4.0, 5.0, 6.0])

# Dot product - three equivalent ways
dot1 = np.dot(x, y)
dot2 = x @ y
dot3 = sum(x[i]*y[i] for i in range(len(x))) # manual
print(dot1, dot2, dot3) # all give 32.0

# Geometric interpretation
cos_theta = np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))
theta_deg = np.degrees(np.arccos(cos_theta))
print(f'Angle: {theta_deg:.2f} degrees') # 12.93 degrees

# For unit vectors, dot product = cosine similarity directly
x_u = x / np.linalg.norm(x)
y_u = y / np.linalg.norm(y)
print(x_u @ y_u) # 0.9746 (close to 1 = very similar direction)

# Orthogonal vectors: dot product = 0
a = np.array([1.0, 0.0, 0.0])
b = np.array([0.0, 1.0, 0.0])
print(a @ b) # 0.0
```

Worked example: finding the most similar word

Suppose we have three word embeddings (simplified to 3D for clarity):

king = [0.8, 0.1, 0.6] (normalized to unit length)

queen = [0.7, 0.2, 0.7]

dog = [0.1, 0.9, 0.4]

Query: which word is most similar to 'king'?

$\text{sim}(\text{king}, \text{queen}) = 0.8*0.7 + 0.1*0.2 + 0.6*0.7 = 0.56 + 0.02 + 0.42 = 1.00$

$\text{sim}(\text{king}, \text{dog}) = 0.8*0.1 + 0.1*0.9 + 0.6*0.4 = 0.08 + 0.09 + 0.24 = 0.41$

Answer: 'queen' (score 1.00) is more similar to 'king' than 'dog' (score 0.41).

Exercises 1.2

No.	Difficulty	Exercise
1.2.1	Starter	Compute the dot product of [1,0,0] with each of: [1,0,0], [0,1,0], [-1,0,0]. What do you notice?
1.2.2	Starter	Write a function dot_product(x, y) that computes the dot product using only a loop and addition — no np.dot or @. Test it against np.dot on random vectors.
1.2.3	Standard	Generate 1000 pairs of random unit vectors in 128 dimensions. Compute the dot product of each pair. Plot a histogram. What is the mean? What distribution does it look like?
1.2.4	Standard	Given five random unit vectors in 64 dimensions and a query vector q, find the one with the highest dot product with q. This is the core operation of a vector database.
1.2.5	Standard	Verify the formula $x \cdot y = \ x\ * \ y\ * \cos(\theta)$ numerically: compute the dot product both ways and confirm they match. Do this for 10 random pairs.
1.2.6	Challenge	TurboQuant's distortion for inner products at b=1 bit is approximately 1.57/d. Generate 5000 pairs of random 512-dim unit vectors (x, y). For each pair, add a small random perturbation of size $\sqrt{1.57/512}$ to x, then compute $\langle y, x_{\text{perturbed}} \rangle - \langle y, x \rangle$. Plot the distribution of these errors. Does its variance match the theoretical value?

Lesson 1.3 Matrices

Transformations that operate on vectors

Estimated time: 4-5 hours

The concept

A matrix is a 2D array of numbers. Think of it as a function that takes a vector as input and produces a new vector as output. When you multiply a matrix M by a vector x , you get a transformed vector $y = M @ x$.

Every layer in a neural network is a matrix multiplication. When a transformer projects a token into query, key, and value spaces, those are matrix multiplications. When TurboQuant rotates your vector, that is a matrix multiplication.

The math

Definition: Matrix

A matrix A in $\mathbb{R}^{m \times n}$ is an array of m rows and n columns of real numbers. $A_{\{i,j\}}$ is the element in row i , column j .

Matrix-vector multiplication

Multiplying an $(m \times n)$ matrix A by an n -dimensional vector x produces an m -dimensional vector y :

$$y = A @ x$$

$$y_i = \sum_{j=1}^n A_{\{i,j\}} * x_j \quad (\text{the dot product of row } i \text{ with } x)$$

Example: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $x = [5, 6]$

$$y_1 = 1*5 + 2*6 = 5 + 12 = 17$$

$$y_2 = 3*5 + 4*6 = 15 + 24 = 39$$

$$y = [17, 39]$$

Matrix-matrix multiplication

Multiplying an $(m \times k)$ matrix A by a $(k \times n)$ matrix B gives an $(m \times n)$ matrix C :

$$C = A @ B$$

$$C_{\{i,j\}} = \sum_{l=1}^k A_{\{i,l\}} * B_{\{l,j\}}$$

Shapes: $(m \times k) @ (k \times n) \rightarrow (m \times n)$

The inner dimensions MUST match.

Key property: matrix multiplication is NOT commutative — $A @ B \neq B @ A$ in general

Transpose

Definition: Transpose

The transpose of matrix A (written A^T) swaps rows and columns: $A^T_{\{i,j\}} = A_{\{j,i\}}$. If A is $(m \times n)$, then A^T is $(n \times m)$.

The identity matrix

Definition: Identity matrix

The identity matrix I_n is the $n \times n$ matrix with 1s on the diagonal and 0s elsewhere. For any matrix A : $A @ I = I @ A = A$. It is the matrix equivalent of the number 1.

Matrix inverse

Definition: Inverse

The inverse of a square matrix A (written A^{-1}) satisfies $A @ A^{-1} = A^{-1} @ A = I$. Not all matrices have an inverse. Those that do are called invertible or non-singular.

Python: matrix operations

```
import numpy as np

# Create a matrix
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A.shape) # (2, 3) -- 2 rows, 3 columns

# Matrix-vector multiply
x = np.array([1.0, 2.0, 3.0])
y = A @ x
print(y) # [14. 32.]

# Manual: row 0 dot x = 1*1 + 2*2 + 3*3 = 1+4+9 = 14

# Matrix-matrix multiply
B = np.array([[1, 0],
              [0, 1],
              [1, 1]])
C = A @ B # (2x3) @ (3x2) -> (2x2)
print(C.shape) # (2, 2)

# Transpose
At = A.T
```

```

print(At.shape)  # (3, 2)

# Identity and inverse
M = np.array([[2.0, 1.0], [5.0, 3.0]])
I = np.eye(2)      # 2x2 identity
M_inv = np.linalg.inv(M)
print(np.allclose(M @ M_inv, I))  # True

# Shapes must be compatible for multiplication
# (3x4) @ (4x5) -> (3x5)   OK
# (3x4) @ (3x5) -> ERROR: inner dims don't match

```

Geometric interpretation: what does a matrix do to space?

Every matrix describes a linear transformation of space. Different matrices do different things:

- Rotation matrices: rotate all vectors by a fixed angle (no stretching)
- Scaling matrices: stretch or shrink along each axis
- Projection matrices: flatten vectors onto a subspace
- Reflection matrices: flip across an axis

In machine learning, the weight matrices in a neural network learn to transform input vectors into useful representations. A well-trained weight matrix projects inputs into a space where similar examples cluster together.

Worked example: a layer of a neural network

Suppose we have a simple layer with input size 3 and output size 2. The weight matrix W is (2×3) . The bias b is a 2-dimensional vector.

Input: $x = [1.0, 0.5, -0.3]$

Weight: $W = [[0.2, 0.8, -0.5], [0.6, -0.2, 0.9]]$

Bias: $b = [0.1, -0.1]$

Output: $y = W @ x + b$

$$y_1 = 0.2 \cdot 1.0 + 0.8 \cdot 0.5 + (-0.5) \cdot (-0.3) + 0.1 = 0.2 + 0.4 + 0.15 + 0.1 = 0.85$$

$$y_2 = 0.6 \cdot 1.0 + (-0.2) \cdot 0.5 + 0.9 \cdot (-0.3) + (-0.1) = 0.6 - 0.1 - 0.27 - 0.1 = 0.13$$

$y = [0.85, 0.13]$

Exercises 1.3

No.	Difficulty	Exercise
1.3.1	Starter	Create a (3×3) matrix and a 3-element vector. Compute the matrix-vector product by hand, then verify with NumPy.

1.3.2	Starter	Create two matrices: A (4 x 3) and B (3 x 5). Compute $C = A @ B$. What shape is C? Now try $A @ A$ — what happens and why?
1.3.3	Standard	Implement matrix-vector multiplication from scratch using only loops — no NumPy matrix ops. Compare your result to $A @ x$.
1.3.4	Standard	For the matrix $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, compute: its transpose, its inverse, and verify that $M @ M_{\text{inv}} = \text{identity}$. Then compute $M @ M^T$. What do you notice?
1.3.5	Standard	A rotation matrix in 2D for angle theta is: $\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$. Apply it to the vector $[1, 0]$ for theta = 0, 45, 90, 180 degrees. Plot the original and all rotated vectors.
1.3.6	Challenge	Prove numerically that matrix multiplication is not commutative: generate 100 pairs of random (3x3) matrices and check that $A @ B$ equals $B @ A$. How often are they equal? When can they be equal?

Lesson 1.4 Orthogonal Matrices

Rotations that preserve everything

Estimated time: 3-4 hours

The concept

An orthogonal matrix is a special kind of matrix that rotates vectors without distorting them. It preserves lengths, angles, and dot products. Think of it as a pure rotation (or reflection) of the coordinate system.

This is the exact type of matrix TurboQuant uses. The very first line of Algorithm 1 is: 'Generate a random rotation matrix P_i '. That matrix is orthogonal.

Why TurboQuant uses an orthogonal matrix

Before quantizing, TurboQuant rotates the input vector: $y = P_i @ x$

After dequantizing, it rotates back: $\hat{x} = P_i^T @ \hat{y}$

This works cleanly because $P_i^T = P_i^{-1}$ for orthogonal matrices.

The rotation does not add or remove information — it just redistributes it.

After rotation, each coordinate follows the same Beta distribution, which allows the same codebook to be applied to every coordinate.

The math

Definition: Orthogonal matrix

A square matrix Q is orthogonal if $Q^T @ Q = Q @ Q^T = I$. This means $Q^{-1} = Q^T$. The rows of Q form an orthonormal basis, and so do the columns.

Key properties — all follow from $Q^T Q = I$

- $Q^{-1} = Q^T$: the inverse is just the transpose (very cheap to compute)
- $\|Qx\| = \|x\|$: orthogonal matrices preserve vector length
- $(Qx) \cdot (Qy) = x \cdot y$: orthogonal matrices preserve dot products
- $\det(Q) = +1$ or -1 : $+1$ means pure rotation, -1 means rotation + reflection

Proof: $\|Qx\| = \|x\|$

$$\begin{aligned}\|Qx\|^2 &= (Qx)^T (Qx) && \text{(by definition of L2 norm squared)} \\ &= x^T Q^T Q x && \text{(transpose of a product reverses order)} \\ &= x^T I x && \text{(because } Q^T Q = I\text{)} \\ &= x^T x \\ &= \|x\|^2\end{aligned}$$

Taking the square root: $\|Qx\| = \|x\|$

Orthonormal basis

The rows (and columns) of an orthogonal matrix form an orthonormal set: every row has length 1 (normal) and every pair of rows is perpendicular (orthogonal). This is why the matrix 'rotates without stretching' — it simply relabels the axes.

Python: orthogonal matrices

```
import numpy as np

# Create a simple 2D rotation matrix
theta = np.pi / 4 # 45 degrees
Q = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])

# Verify Q^T Q = I
print(np.allclose(Q.T @ Q, np.eye(2))) # True

# Verify norm preservation
x = np.array([3.0, 4.0])
y = Q @ x
print(np.linalg.norm(x)) # 5.0
print(np.linalg.norm(y)) # 5.0 (same!)

# Verify dot product preservation
a = np.array([1.0, 0.0])
b = np.array([0.0, 1.0])
Qa = Q @ a
Qb = Q @ b
print(a @ b) # 0.0
print(Qa @ Qb) # 0.0 (preserved!)

# Verify inverse = transpose
print(np.allclose(np.linalg.inv(Q), Q.T)) # True
```

Worked example: rotation in 2D

Let Q be the 90-degree rotation matrix: $Q = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

Verify: $Q^T Q = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} @ \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$. Yes, orthogonal.

Apply to $x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$: $Q @ \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0*1 + (-1)*0 \\ 1*1 + 0*0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

The vector pointing right now points up. That is a 90-degree rotation.

The norm is preserved: $\|\begin{bmatrix} 1 \\ 0 \end{bmatrix}\| = 1 = \|\begin{bmatrix} 0 \\ 1 \end{bmatrix}\|$.

Exercises 1.4

No.	Difficulty	Exercise
1.4.1	Starter	Verify that the 45-degree rotation matrix (from the Python example) satisfies $Q^T Q = I$ and $Q @ Q^T = I$.
1.4.2	Starter	Apply the 45-degree rotation matrix to 8 vectors evenly spaced on a unit circle. Plot the original and rotated vectors. What do you observe?
1.4.3	Standard	Generate a random 4x4 matrix M . Compute $Q, R = \text{np.linalg.qr}(M)$. Verify that Q is orthogonal: check $Q^T @ Q$ and $Q @ Q^T$ are both close to identity.
1.4.4	Standard	Take a vector x and an orthogonal matrix Q . Verify numerically that: (1) $\ Q @ x\ = \ x\ $, (2) the dot product is preserved for two vectors. Use $d=128$ and random Q from QR decomposition.
1.4.5	Standard	TurboQuant dequantization uses Q^T . Generate a random orthogonal Q (64x64). Rotate a vector x with Q . Rotate back with Q^T . Verify you recover x exactly (within floating-point precision).
1.4.6	Challenge	In 2D, list all 2x2 orthogonal matrices. There are two families: rotations and reflections. What distinguishes them? Verify your answer using the determinant ($\det = +1$ vs -1). Write code to classify any 2x2 orthogonal matrix as a rotation or reflection.

Lesson 1.5 QR Decomposition

How to build a random rotation matrix

Estimated time: 2-3 hours

The concept

QR decomposition factors any matrix A into: $A = Q @ R$, where Q is orthogonal and R is upper triangular (zeros below the diagonal). It is one of the fundamental algorithms in numerical linear algebra.

For TurboQuant, QR decomposition has exactly one purpose: generating a random orthogonal matrix Q . The recipe is: fill a matrix with random Gaussian numbers, then call QR decomposition. The Q factor is a uniformly random orthogonal matrix.

The two uses of QR decomposition

1. Solving linear systems and least-squares problems (important in ML, not our focus here)
2. Generating random orthogonal matrices (the TurboQuant use case)

We focus on use case 2. Once you understand that a Gaussian matrix's Q factor is a Haar-random orthogonal matrix (uniformly random over all orthogonal matrices), you have everything you need to understand TurboQuant's rotation step.

The math

Definition: QR decomposition

Any real matrix A in $R^{m \times n}$ with $m \geq n$ can be written as $A = Q @ R$, where Q in $R^{m \times n}$ has orthonormal columns ($Q^T Q = I$), and R in $R^{n \times n}$ is upper triangular with positive diagonal entries.

Why random Gaussian -> uniform random orthogonal?

If G is a matrix whose entries are i.i.d. samples from $N(0,1)$ (the standard normal distribution), then the Q factor from $QR(G)$ is uniformly distributed over all orthogonal matrices. This distribution is called the Haar measure.

Intuitively: the Gaussian distribution is spherically symmetric (it does not prefer any direction), so a matrix of Gaussian entries has no preferred orientation, and its orthogonal factor covers all orientations equally.

TurboQuant rotation matrix recipe

1. Generate G : a $d \times d$ matrix with $G_{\{i,j\}} \sim N(0,1)$ independently

2. Compute $Q, R = QR(G)$
3. Q is now a uniformly random orthogonal matrix

This is exactly what the paper's Algorithm 1 line 2 describes:
'Generate a random rotation matrix P_i in $R^{d \times d}$ '

Python: QR decomposition

```
import numpy as np

# QR decomposition of a small matrix
A = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0],
              [7.0, 8.0, 9.0]])

Q, R = np.linalg.qr(A)
print('Q shape:', Q.shape)  # (3, 3)
print('R shape:', R.shape)  # (3, 3)

# Verify Q is orthogonal
print(np.allclose(Q.T @ Q, np.eye(3)))  # True

# Verify reconstruction: A = Q @ R
print(np.allclose(A, Q @ R))  # True

# Generating a random orthogonal matrix for TurboQuant
d = 512
G = np.random.randn(d, d)  # Gaussian entries
Q, _ = np.linalg.qr(G)  # Only need Q

# Verify it is truly orthogonal
print(np.allclose(Q.T @ Q, np.eye(d), atol=1e-5))  # True
print(np.allclose(Q @ Q.T, np.eye(d), atol=1e-5))  # True

# Rotate a unit vector and verify norm is preserved
x = np.random.randn(d)
x = x / np.linalg.norm(x)
y = Q @ x
print(f'||x|| = {np.linalg.norm(x):.6f}')  # 1.000000
print(f'||Qx|| = {np.linalg.norm(y):.6f}')  # 1.000000
```

Worked example: verify the rotation step of TurboQuant

This implements the core rotation from Algorithm 1 of the TurboQuant paper and verifies its key properties:

```
import numpy as np

np.random.seed(42)
```

```

d = 256

# Step 1: generate the random rotation matrix
G = np.random.randn(d, d)
Pi, _ = np.linalg.qr(G)

# Step 2: create a worst-case input vector (adversarial: all mass on one coord)
x = np.zeros(d)
x[0] = 1.0 # all mass in one coordinate

# Step 3: rotate
y = Pi @ x

# Before rotation: x[0]=1.0, all others=0
print('Before: max coord =', x.max(), ', min =', x.min())

# After rotation: coordinates spread out evenly
print('After: max coord =', y.max().round(4), ', std =', y.std().round(4))

# The std after rotation should be approx 1/sqrt(d) = 1/sqrt(256) = 0.0625
print('Expected std:', 1/np.sqrt(d))

# Key: the norm is unchanged
print('||x|| =', np.linalg.norm(x))
print('||y|| =', np.linalg.norm(y))

```

Exercises 1.5

No.	Difficulty	Exercise
1.5.1	Starter	Call <code>np.linalg.qr</code> on a 3x3 random matrix. Print Q and R. Verify: $Q^T @ Q = I$, and $Q @ R =$ original matrix.
1.5.2	Standard	Generate 10 random orthogonal matrices of size 64x64 using the Gaussian + QR recipe. For each, verify it is orthogonal. Measure the time it takes.
1.5.3	Standard	Take the 'adversarial' vector from the worked example (all mass on one coordinate). After rotation, plot the distribution of coordinates. Does it look like a Gaussian? What is the expected std?
1.5.4	Standard	Demonstrate that two different random seeds give different rotation matrices, but both are valid (orthogonal). Compute the angle between the two rotated versions of the same input vector.
1.5.5	Challenge	Timing experiment: measure how long it takes to generate a random orthogonal matrix using QR for $d = 64, 128, 256, 512, 1024$. Plot time vs d . How does it scale? (Hint: QR decomposition is $O(d^3)$. Does your plot confirm this?)
1.5.6	Challenge	The TurboQuant paper says the rotation is applied once and stored (it is a 'global parameter', not recomputed for each vector). Why is this important? How much memory does storing Q require for $d=1536$? Could you get a

		similar effect with a faster operation? Look up 'Hadamard transform' and 'SRHT' — sketch the idea.
--	--	--

Lesson 1.6 Eigenvalues & Eigenvectors

The natural axes of a transformation

Estimated time: 3-4 hours

The concept

When you multiply most vectors by a matrix, they get rotated and stretched. But some special vectors only get scaled — they point in the same direction as before. These are called eigenvectors, and the scale factor is the eigenvalue.

Eigenvectors reveal the 'natural axes' of a transformation — the directions the matrix cares about most. In PCA (used in ML for dimensionality reduction), eigenvectors of the covariance matrix are the principal components.

Where eigenvalues appear in ML

PCA: eigenvectors of the covariance matrix give the principal components

Spectral methods for graph neural networks

Understanding why certain weight initializations train better (via spectral radius)

Hessian analysis: eigenvalues of the loss Hessian describe the loss landscape shape

Stable training: ensuring the spectral radius of weight matrices is controlled

The math

Definition: Eigenvalue and eigenvector

For a square matrix A , a non-zero vector v is an eigenvector with eigenvalue λ if: $Av = \lambda v$. The matrix A only scales v (by factor λ) rather than rotating it.

Finding eigenvalues

Eigenvalues are the solutions to the characteristic equation: $\det(A - \lambda I) = 0$. This gives a polynomial in λ whose roots are the eigenvalues.

Example: $A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$. Characteristic equation: $\det(\begin{bmatrix} 2-\lambda & 1 \\ 0 & 3-\lambda \end{bmatrix}) = (2-\lambda)(3-\lambda) = 0$. Eigenvalues: $\lambda = 2$ and $\lambda = 3$.

Properties of symmetric matrices

Symmetric matrices ($A = A^T$) have two beautiful properties that are used throughout ML:

- All eigenvalues are real numbers
- Eigenvectors corresponding to different eigenvalues are orthogonal
- They can always be decomposed as $A = V \Lambda V^T$, where V is orthogonal and Λ is diagonal

Eigendecomposition

Definition: Eigendecomposition

A diagonalizable matrix A can be written as $A = V @ \text{Lambda} @ V^{-1}$, where V is the matrix of eigenvectors (columns) and Lambda is a diagonal matrix of eigenvalues.

Python: eigenvalues and eigenvectors

```
import numpy as np

# Simple 2x2 matrix
A = np.array([[4.0, 1.0],
              [2.0, 3.0]])

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print('Eigenvalues:', eigenvalues)    # [5. 2.]
print('Eigenvectors:')                # columns are eigenvectors
print(eigenvectors)

# Verify: A @ v = lambda * v for each eigenpair
for i in range(len(eigenvalues)):
    v = eigenvectors[:, i]            # i-th column
    lam = eigenvalues[i]
    lhs = A @ v
    rhs = lam * v
    print(f'lambda={lam:.1f}: match = {np.allclose(lhs, rhs)}')

# For symmetric matrices, use eigh (more numerically stable)
S = A + A.T # make symmetric
lam_s, V_s = np.linalg.eigh(S)

# Verify eigendecomposition: S = V @ diag(lam) @ V^T
S_reconstructed = V_s @ np.diag(lam_s) @ V_s.T
print(np.allclose(S, S_reconstructed)) # True

# Covariance matrix example (used in PCA)
X = np.random.randn(100, 5) # 100 samples, 5 features
cov = X.T @ X / 100         # 5x5 covariance matrix
lam, V = np.linalg.eigh(cov)
print('Eigenvalues (variance explained):', lam[::-1].round(2))
```

Worked example: PCA intuition

PCA finds the directions of maximum variance in a dataset. These are the eigenvectors of the covariance matrix with the largest eigenvalues.

```
import numpy as np
```

```

np.random.seed(0)

# Create data with strong structure in one direction
n = 200
t = np.random.randn(n)          # 1D latent variable
X = np.column_stack([
    3*t + 0.3*np.random.randn(n), # strongly correlated with t
    1*t + 0.3*np.random.randn(n)  # less strongly correlated
])

# Compute covariance matrix
X_centered = X - X.mean(axis=0)
cov = X_centered.T @ X_centered / n
print('Covariance matrix:')
print(cov.round(2))

# Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eigh(cov)
idx = np.argsort(eigenvalues)[::-1] # sort descending
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

print('Eigenvalues:', eigenvalues.round(2))
print('Principal component 1:', eigenvectors[:,0].round(3))
# This vector points along the direction of most variance

```

Exercises 1.6

No.	Difficulty	Exercise
1.6.1	Starter	For the matrix $\begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix}$, find the eigenvalues and eigenvectors by inspection (no computation needed). Then verify with <code>np.linalg.eig</code> .
1.6.2	Starter	Verify the definition $A \cdot v = \lambda \cdot v$ for all eigenpairs of $\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$. Use <code>np.linalg.eig</code> to get the pairs.
1.6.3	Standard	Generate a random 5x5 symmetric matrix ($S = A + A^T$ for random A). Compute its eigendecomposition. Verify: (1) all eigenvalues are real, (2) eigenvectors are orthogonal, (3) $S = V @ \text{diag}(\lambda) @ V^T$.
1.6.4	Standard	Generate 50 samples from a 2D Gaussian with covariance $\begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix}$. Compute the sample covariance matrix. Find its eigenvectors. Plot the data and overlay the eigenvectors scaled by the square root of the eigenvalues.
1.6.5	Standard	Implement PCA from scratch using eigendecomposition: center the data, compute the covariance matrix, find the top-k eigenvectors, project the data. Compare to <code>sklearn.decomposition.PCA</code> .
1.6.6	Challenge	The spectral radius of a matrix is its largest absolute eigenvalue. Generate random weight matrices of size 100x100 and compute their spectral radius. If you scale each matrix to have spectral radius exactly 1.0, what happens when you repeatedly multiply a vector by it? What if the spectral radius is 0.9? 1.1? (Hint: this is related to why careful weight initialization matters in deep networks.)

Capstone Project

Build a mini TurboQuant — the rotation + codebook pipeline from scratch

This project ties together every lesson in this workbook. You will implement the core of TurboQuant's Algorithm 1 from scratch, using only NumPy, with no reference to the course code. Then you will measure how well it works.

What you are building

A simple vector quantizer that:

1. Rotates an input vector using a random orthogonal matrix
2. Quantizes each coordinate to the nearest value in a codebook
3. Dequantizes by looking up centroids and rotating back
4. Measures how much distortion was introduced

Part A: Setup (30 mins)

Write the following functions. Each one corresponds to a concept from the lessons:

1. `generate_rotation(d, seed)` — generates a $d \times d$ random orthogonal matrix using QR decomposition (Lesson 1.5)
2. `normalize(x)` — normalizes a vector to unit L2 norm (Lesson 1.1)
3. `rotate(Q, x)` — applies the rotation matrix to a vector (Lesson 1.4)
4. `unrotate(Q, y)` — applies Q^T to reverse the rotation (Lesson 1.4)

Part B: Simple codebook (30 mins)

For a bit-width of b bits, your codebook has 2^b centroids. Use a simple approach for now: place the centroids evenly between $-1/\sqrt{d}$ and $+1/\sqrt{d}$ (the approximate range of coordinates after rotation).

5. `build_codebook(n_levels, d)` — returns an array of n_levels evenly spaced centroids
6. `quantize_coordinate(value, codebook)` — returns the index of the nearest centroid
7. `dequantize_coordinate(index, codebook)` — returns the centroid value

Part C: Full pipeline (30 mins)

Combine everything into two functions:

8. `compress(x, Q, codebook)` -> indices — normalize, rotate, quantize each coordinate
9. `decompress(indices, Q, codebook, norm)` -> $\hat{x}$ — dequantize, unrotate, rescale

Part D: Measure distortion (30 mins)

Generate 1000 random unit vectors in 256 dimensions. For each vector: compress then decompress it. Measure:

- MSE distortion: $E[\|x - \hat{x}\|^2]$
- Inner product distortion: compare $\langle y, x \rangle$ vs $\langle y, \hat{x} \rangle$ for a fixed query vector y

Try bit-widths $b = 1, 2, 3, 4$. How does distortion change? Does it follow the 4^{-b} pattern from the TurboQuant paper?

Part E: Compare against the theoretical bound (30 mins)

The paper's lower bound on MSE distortion is 4^{-b} . Compute 4^{-b} for $b = 1, 2, 3, 4$ and compare to your empirical measurements. Create a table:

b	Your MSE	Lower bound 4^{-b}	Ratio	Within 2.7x?
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____

You have completed the capstone when

- ☐ All four functions (generate_rotation, normalize, rotate, unrotate) pass basic tests
- ☐ Your compress/decompress pipeline recovers vectors with reasonable MSE
- ☐ Your MSE at $b=4$ is less than 0.02 (it should be around 0.009-0.015)
- ☐ Your distortion decreases by roughly 4x for each extra bit
- ☐ You can explain every step in plain English to someone who has not read this workbook

Course completion checklist

Go through this before moving to Lesson 1.2 in the roadmap (The Dot Product is already covered here). Use it before starting Layer 2.

Vectors (Lesson 1.1)

- I can create, add, and scale vectors in NumPy
- I can compute the L2 norm manually and with `np.linalg.norm`
- I can normalize any vector to unit length
- I understand why TurboQuant requires unit-norm inputs

Dot product (Lesson 1.2)

- I can compute the dot product manually and verify with NumPy
- I can explain the geometric interpretation: $x \cdot y = \|x\| \|y\| \cos(\theta)$
- I know that for unit vectors, the dot product equals the cosine of the angle
- I understand why preserving dot products matters for vector search

Matrices (Lesson 1.3)

- I can multiply a matrix by a vector and by another matrix
- I understand what shapes are valid for multiplication
- I can compute the transpose and inverse of a matrix
- I understand that a neural network layer is a matrix multiplication

Orthogonal matrices (Lesson 1.4)

- I know the definition: $Q^T Q = I$
- I can prove that orthogonal matrices preserve norms and dot products
- I understand why $Q^{-1} = Q^T$ is computationally useful
- I can verify that a given matrix is orthogonal

QR decomposition (Lesson 1.5)

- I can generate a random orthogonal matrix using `np.linalg.qr`
- I understand why a Gaussian matrix's Q factor is uniformly random
- I can implement the rotation step of TurboQuant Algorithm 1

Eigenvalues & eigenvectors (Lesson 1.6)

- I know the definition: $A \cdot v = \lambda \cdot v$

- I can compute eigendecomposition with `np.linalg.eig` and `eigh`
- I understand that symmetric matrices have orthogonal eigenvectors
- I can explain the connection between eigenvalues and PCA

Ready for Layer 2 when

You can answer all questions in the checklist above without referring to notes.

Your capstone project produces distortion values within the theoretical bounds.

You can open the TurboQuant paper, read Algorithm 1, and follow every single line.